

# Introducción a Supabase

## Bases de datos para automatizaciones con n8n

Una automatización no siempre alcanza con recibir un dato y ejecutar una acción inmediata. En muchos casos, también necesita guardar información para consultarla, actualizarla o usarla más adelante.

**Una base de datos permite que el *workflow* tenga memoria operativa.** Puede servir para guardar:

- *leads* o contactos;
- tickets o solicitudes;
- conversaciones con IA;

- estados de procesos;
- métricas de negocio;
- historial de acciones;
- registros de ejecución;
- archivos o referencias a documentos.

### Ejemplo breve

Un formulario puede enviar datos a n8n. Luego, n8n puede guardar esos datos en una base para que el equipo comercial los consulte, los actualice o los use en futuras automatizaciones.

## Datos temporales y datos persistentes

En un *workflow*, algunos datos se usan solo durante una ejecución puntual. Por ejemplo, cuando una persona completa un formulario, n8n recibe su nombre, *email*, empresa y necesidad. Con esa información puede enviar un correo, notificar a un equipo o actualizar una herramienta.

Pero, si esa información se necesita más adelante, no alcanza con procesarla en el momento: hay que guardarla en una base de datos.

Los **datos temporales** sirven para ejecutar una acción inmediata. Los **datos persistentes** quedan almacenados para poder consultarlos, actualizarlos o reutilizarlos en otros procesos.

### Un ejemplo breve

Un *lead* recibido desde una *landing* puede usarse en el momento para enviar una notificación a ventas. Pero si el equipo necesita hacer seguimiento, cambiar su estado a “contactado” o consultar su historial, ese *lead* debe guardarse en una base de datos.

## Qué puede guardar n8n en una base de datos

| Caso de uso             | Qué se guarda                  | Para qué sirve           |
|-------------------------|--------------------------------|--------------------------|
| Captura de <i>leads</i> | Nombre, email, empresa, estado | Seguimiento comercial    |
| Soporte                 | Ticket, prioridad, responsable | Gestión de atención      |
| Chat con IA             | Pregunta, respuesta, usuario   | Historial conversacional |
| <i>Dashboard</i>        | Métrica, valor, fecha          | Visualización y análisis |
| Procesos internos       | Solicitud, estado, fecha       | Control operativo        |

## ¿Qué es Supabase?

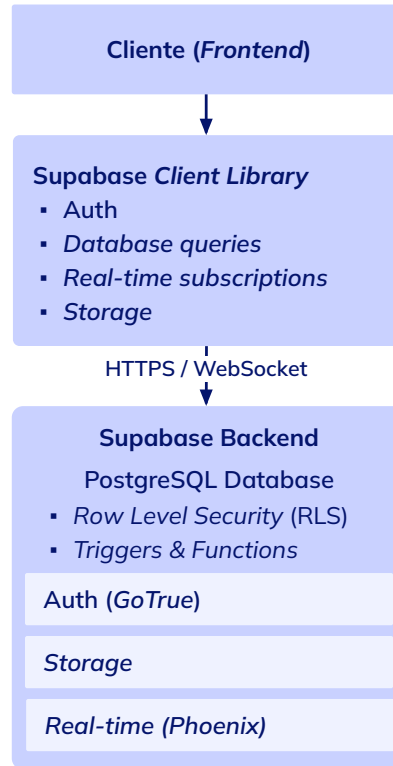
Supabase es una plataforma que permite **crear y administrar una base de datos en la nube**. Entre sus herramientas, ofrece:

- base de datos PostgreSQL;
- autenticación de usuarios;
- almacenamiento de archivos;
- actualizaciones en tiempo real;
- APIs automáticas;
- integración con aplicaciones e interfaces.

Supabase puede conectarse con n8n y, en una solución automatizada, cumple tres funciones principales:

1. Guardar datos recibidos desde formularios o *workflows*.
2. Permitir consultas para recuperar información cuando el *workflow* la necesite.
3. Actualizar registros cuando cambia el estado de un proceso.

## Arquitectura de Supabase



# Tutorial completo de Supabase

## Paso 1: Crear Proyecto en Supabase

1. Visita [supabase.com](https://supabase.com).
2. Click en "*Start your project*".
3. Crea una organización (ej: "Mi Empresa").
4. Crea un nuevo proyecto:
  - a. Name: "*n8n-leads-system*".
  - b. *Database Password*: (genera uno seguro).
  - c. Región: Selecciona más cercana a tus usuarios.
5. Espera ~2 minutos mientras se aprovisiona.



## Paso 2: Crear Esquema de Base de Datos

### Ejemplo de esquema: tabla de *leads*

El ejemplo muestra una estructura posible para guardar *leads* en Supabase. El código crea:

- una tabla principal de *leads*;
- campos de contacto y empresa;
- un estado comercial;
- índices para facilitar consultas;
- una tabla de actividades;
- una vista para consultar *leads* con actividad asociada.

| Elemento del esquema      | Uso desde n8n                                       |
|---------------------------|-----------------------------------------------------|
| email                     | Identificar si el <i>lead</i> ya existe             |
| status                    | Saber si está nuevo, contactado o calificado        |
| source                    | Registrar desde dónde llegó                         |
| created_at                | Ordenar o filtrar por fecha                         |
| lead_activities           | Guardar llamadas, <i>emails</i> , reuniones o notas |
| leads_with_activity_count | Consultar <i>leads</i> con información resumida     |

## Opción A: SQL Editor (recomendado para aprender)

En Supabase *dashboard*, ve a *SQL Editor* y ejecuta:

```
-- Crear tabla de leads
CREATE TABLE leads (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  -- Información personal
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
```



...

```
phone TEXT,  
-- Información de empresa  
company_name TEXT,  
company_size TEXT,  
industry TEXT,  
-- Necesidades  
goals TEXT[], -- Array de strings  
budget TEXT,  
timeline TEXT,  
-- Metadata  
source TEXT DEFAULT 'direct',  
status TEXT DEFAULT 'nuevo' CHECK (status IN ('nuevo', 'contactado', 'calificado', 'ganado', 'perdido')),  
assigned_to UUID REFERENCES auth.users(id),  
notes TEXT  
);  
-- Índices para búsquedas rápidas  
CREATE INDEX idx_leads_email ON leads(email);  
CREATE INDEX idx_leads_status ON leads(status);
```

...

...

```
CREATE INDEX idx_leads_created_at ON leads(created_at DESC);
-- Trigger para actualizar updated_at automáticamente
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ language 'plpgsql';
CREATE TRIGGER update_leads_updated_at
BEFORE UPDATE ON leads
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();
-- Tabla de actividades (log de interacciones)
CREATE TABLE lead_activities (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    lead_id UUID NOT NULL REFERENCES leads(id) ON DELETE CASCADE,
    activity_type TEXT NOT NULL, -- 'email', 'call', 'meeting', 'note'
```

...

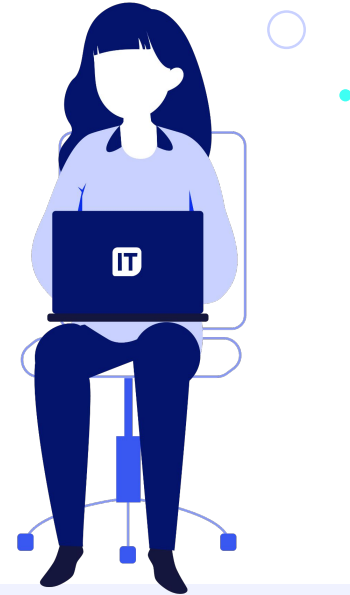
...

```
description TEXT,  
performed_by UUID REFERENCES auth.users(id)  
);  
CREATE INDEX idx_activities_lead_id ON lead_activities(lead_id);  
-- Vista para leads con conteo de actividades  
CREATE VIEW leads_with_activity_count AS  
SELECT  
  l.*,  
  COUNT(a.id) as activity_count,  
  MAX(a.created_at) as last_activity_at  
FROM leads l  
LEFT JOIN lead_activities a ON l.id = a.lead_id  
GROUP BY l.id;
```

## Opción B: *Table Editor* (visual)

Supabase también permite crear tablas desde una interfaz visual. Esta opción es útil cuando se quiere comprender la estructura de datos sin escribir SQL desde cero.

1. Ve a "*Table Editor*" en *sidebar*.
2. Click "*New Table*".
3. Configura columnas visualmente.
4. Supabase genera el SQL automáticamente.



### Paso 3: Configurar Row Level Security (RLS)

RLS asegura que los usuarios solo puedan acceder a sus propios datos:

```
-- Habilitar RLS
ALTER TABLE leads ENABLE ROW LEVEL SECURITY;
ALTER TABLE lead_activities ENABLE ROW LEVEL SECURITY;
-- Política: Cualquiera puede insertar leads (formulario público)
CREATE POLICY "Cualquiera puede crear leads"
  ON leads FOR INSERT
  TO anon
  WITH CHECK (true);
-- Política: Solo usuarios autenticados pueden ver leads
CREATE POLICY "Usuarios autenticados pueden ver todos los leads"
  ON leads FOR SELECT
  TO authenticated
  USING (true);
-- Política: Solo usuarios autenticados pueden actualizar
```

...

```
CREATE POLICY "Usuarios autenticados pueden actualizar leads"
  ON leads FOR UPDATE
  TO authenticated
  USING (true);
-- Políticas similares para activities
CREATE POLICY "Usuarios autenticados pueden ver activities"
  ON lead_activities FOR SELECT
  TO authenticated
  USING (true);
CREATE POLICY "Usuarios autenticados pueden crear activities"
  ON lead_activities FOR INSERT
  TO authenticated
  WITH CHECK (true);
```

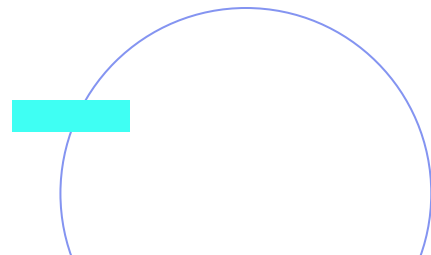
## Paso 4: Configurar Autenticación

### Habilitar Proveedores de Auth:

1. Ve a *Authentication > Providers*.
2. Habilita los proveedores que necesites:
  - a. *Email* (habilitado por defecto).
  - b. Google.
  - c. GitHub.
  - d. etc.

### Configurar *Email Templates*:

1. *Authentication > Email Templates*.
2. Personaliza *templates* de:
  - a. Confirmación de *email*.
  - b. *Reset* de contraseña.
  - c. Cambio de *email*.



## Crear Usuario de Prueba:

- En SQL Editor, insertar usuario de prueba.
- Nota: En producción, usuarios se crean via *signup*.

```
INSERT INTO auth.users (  
  id,  
  email,  
  encrypted_password,  
  email_confirmed_at,  
  created_at,  
  updated_at  
) VALUES (  
  gen_random_uuid(),  
  'test@example.com',  
  crypt('password123', gen_salt('bf')),  
  NOW(),  
  NOW(),  
  NOW()  
);
```

## Dos formas de usar Supabase en una solución

Supabase puede conectarse con distintos componentes de una solución automatizada.

| Conexión                  | Para qué sirve                                               | Ejemplo                                           |
|---------------------------|--------------------------------------------------------------|---------------------------------------------------|
| Interfaz ↔ Supabase       | Mostrar o cargar datos directamente desde una UI             | <i>Dashboard</i> que consulta <i>leads</i>        |
| n8n ↔ Supabase            | Guardar, consultar o actualizar datos desde <i>workflows</i> | <i>Webhook</i> que guarda un <i>lead</i>          |
| Interfaz ↔ n8n ↔ Supabase | Mantener n8n como coordinador del proceso                    | Formulario que envía datos a n8n y n8n los guarda |

## Paso 5: Integrar Supabase en *Frontend*

En algunos casos, una interfaz puede conectarse directamente con Supabase para consultar o mostrar datos.

### Instalar Cliente de Supabase:

```
npm install @supabase/supabase-js
```

### Configurar Cliente:

```
// lib/supabase.ts
import { createClient } from '@supabase/supabase-js'
const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL!
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
export const supabase = createClient(supabaseUrl, supabaseAnonKey)
// Tipos para TypeScript
export type Lead = {
  id: string
```

...

```
created_at: string
updated_at: string
first_name: string
last_name: string
email: string
phone?: string
company_name?: string
company_size?: string
industry?: string
goals?: string[]
budget?: string
timeline?: string
source: string
status: 'nuevo' | 'contactado' | 'calificado' | 'ganado' | 'perdido'
assigned_to?: string
notes?: string
}
```

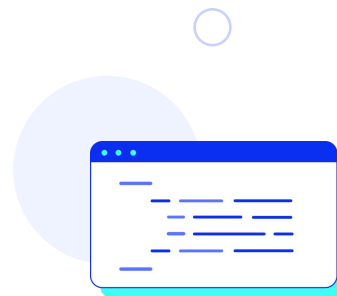
## Obtener credenciales:

1. En Supabase *dashboard*: *Settings* > API
2. Copia:
  - *Project URL*: NEXT\_PUBLIC\_SUPABASE\_URL
  - *anon public key*:  
NEXT\_PUBLIC\_SUPABASE\_ANON\_KEY
  - *service\_role secret*:  
SUPABASE\_SERVICE\_ROLE\_KEY (solo *backend*)

## Variables de Entorno:

# .env.local

```
NEXT_PUBLIC_SUPABASE_URL=https://tu-proyecto.supabase.co  
NEXT_PUBLIC_SUPABASE_ANON_KEY=tu-anon-key-aqui  
SUPABASE_SERVICE_ROLE_KEY=tu-service-role-key-aqui
```



## Paso 6: Operaciones CRUD con Supabase

### Crear (*Insert*):

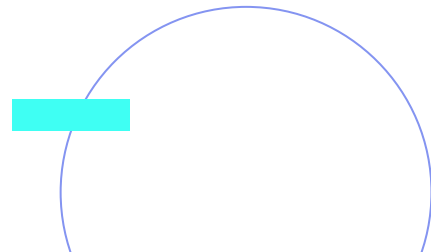
```
// Insertar un nuevo lead
const { data, error } = await supabase
  .from('leads')
  .insert([
    {
      first_name: 'Juan',
      last_name: 'Pérez',
      email: 'juan@example.com',
      company_name: 'Tech Corp',
      status: 'nuevo',
      source: 'website'
    }
  ])
  .select()
if (error) {
  console.error('Error insertando lead:', error)
} else {
  console.log('Lead creado:', data)
}
```

## Leer (Select):

```
// Obtener todos los leads
const { data: leads, error } = await supabase
  .from('leads')
  .select('*')
  .order('created_at', { ascending: false })
// Con filtros
const { data: newLeads } = await supabase
  .from('leads')
  .select('*')
  .eq('status', 'nuevo')
  .gte('created_at', '2025-10-01')
// Con joins
const { data: leadsWithActivities } = await supabase
  .from('leads')
  .select(`
    *,
    lead_activities (
```

...

```
    id,  
    activity_type,  
    description,  
    created_at  
  )  
`)  
.eq('id', leadId)  
.single()
```



## Actualizar (*Update*):

```
// Actualizar status de lead
const { data, error } = await supabase
  .from('leads')
  .update({ status: 'contactado' })
  .eq('id', leadId)
  .select()
```

## Eliminar (*Delete*):

```
// Eliminar lead
const { error } = await supabase
  .from('leads')
  .delete()
  .eq('id', leadId)
```

## Paso 7: Real-time Subscriptions

Supabase permite escuchar cambios en tiempo real:

```
// Escuchar inserts de nuevos leads
const subscription = supabase
  .channel('leads-changes')
  .on(
    'postgres_changes',
    {
      event: 'INSERT',
      schema: 'public',
      table: 'leads'
    },
    (payload) => {
      console.log('Nuevo lead:', payload.new)
      // Actualizar UI, mostrar notificación, etc.
    }
  )
  .subscribe()
// Cleanup
subscription.unsubscribe()
```

## Ejemplo en React:

```
'use client'
import { useEffect, useState } from 'react'
import { supabase, type Lead } from '@lib/supabase'
export default function LeadsRealtimeDashboard() {
  const [leads, setLeads] = useState<Lead[]>([])
  useEffect(() => {
    // Cargar leads iniciales
    fetchLeads()
    // Suscribirse a cambios
    const subscription = supabase
      .channel('leads-realtime')
      .on(
        'postgres_changes',
        {
          event: '*', // INSERT, UPDATE, DELETE
          schema: 'public',
          table: 'leads'
        },

```

...

```
(payload) => {  
  if (payload.eventType === 'INSERT') {  
    setLeads((current) => [payload.new as Lead, ...current])  
  } else if (payload.eventType === 'UPDATE') {  
    setLeads((current) =>  
      current.map((lead) =>  
        lead.id === payload.new.id ? (payload.new as Lead) : lead  
      )  
    )  
  } else if (payload.eventType ===  
  
'DELETE') {  
    setLeads((current) =>  
      current.filter((lead) => lead.id !== payload.old.id)  
    )  
  }  
}
```

...

...

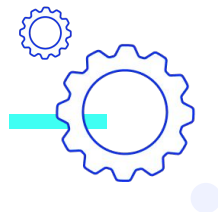
```
)
  .subscribe()
return () => {
  subscription.unsubscribe()
}
}, [])
const fetchLeads = async () => {
  const { data } = await supabase
    .from('leads')
    .select('*')
    .order('created_at', { ascending: false })
    .limit(50)
  if (data) setLeads(data)
}
return (
  <div>
    <h1>Leads en Tiempo Real ({leads.length})</h1>
    { /* Renderizar leads */ }
  </div>
)
}
```

# Configuración de Supabase con n8n

Conectar Supabase con n8n permite que los *workflows* puedan guardar, consultar y actualizar información en una base de datos.

n8n puede trabajar con Supabase de dos maneras principales:

- **Supabase Node:** Operaciones simples de alto nivel (API) sobre tablas, como insertar o consultar registros.
- **Postgres Node:** Consultas SQL más específicas o avanzadas.



## Paso 1: Configurar *Credenciales* de Supabase en n8n

### Credenciales del Supabase *Node*:

1. En n8n, crea nuevo nodo "Supabase".
2. Click en "*Credentials*".
3. Ingresa:
  - a. *Host*: Tu URL de Supabase (ej:  
`https://tu-proyecto.supabase.co`).
  - b. *Service Role Secret*: El `service_role` key de Supabase.

### Obtener Service Role Key:

- Supabase *Dashboard* > *Settings* > *API*.
- Copia `service_role` key (la marked como "secret").



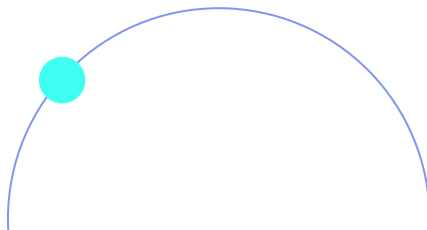
## Paso 2: Configurar Credenciales de Postgres

1. En n8n, crea *nodo* "Postgres".
2. Click en "Credentials".
3. Ingresa:
  - a. *Host*: db.tu-proyecto.supabase.co
  - b. *Database*: postgres
  - c. *User*: postgres
  - d. *Password*: Tu database password (definido al crear proyecto)
  - e. *Port*: 5432
  - f. *SSL*: Activado

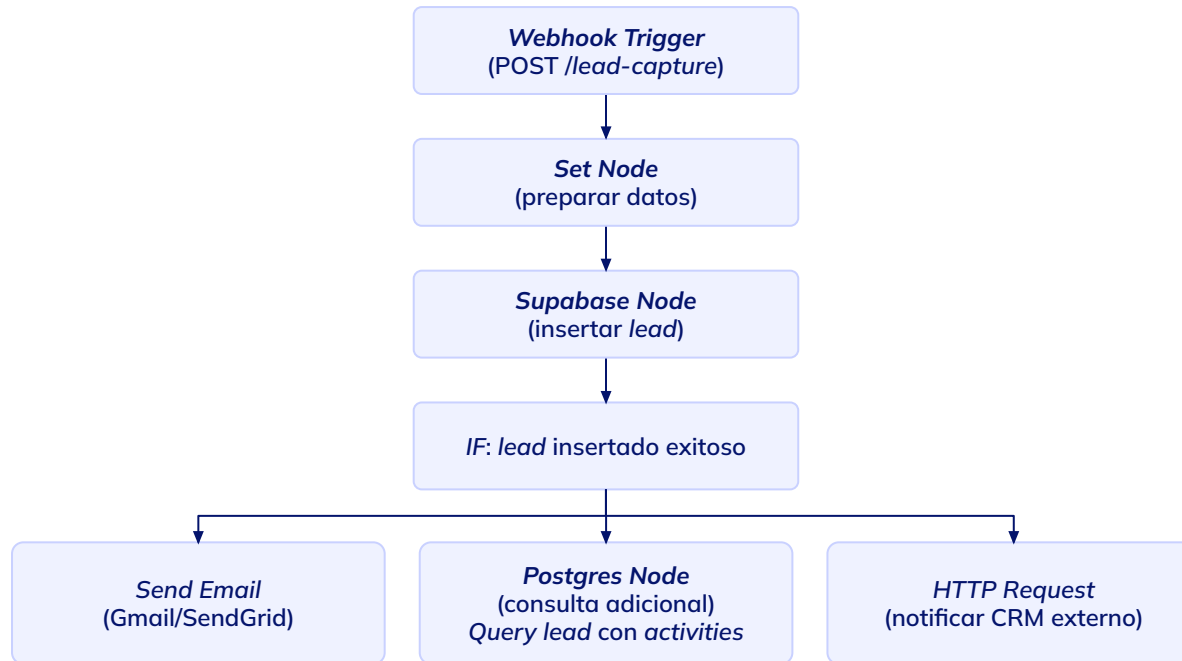
### Obtener *Connection String*:

Supabase Dashboard > Settings > Database > Connection string Formato:

```
postgresql://postgres:[YOUR-PASSWORD]@db.tu-proyecto.  
o.supabase.co:5432/postgres
```



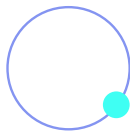
### Paso 3: Workflow de Ejemplo - Captura de *Lead* con Supabase



## Configuración detallada:

### Nodo 1: Webhook Trigger

- *Path: /lead-capture*
- *Method: POST*
- *Response: Immediately*



**Nodo 2: Set (Preparar Datos).** Esto permite mejorar la calidad de la información guardada.

```
// Limpiar y validar datos
return {
  first_name: $json.body.firstName?.trim(),
  last_name: $json.body.lastName?.trim(),
  email: $json.body.email?.toLowerCase().trim(),
  phone: $json.body.phone?.trim(),
  company_name: $json.body.companyName?.trim(),
  company_size: $json.body.companySize,
  industry: $json.body.industry,
  goals: $json.body.goals || [],
  budget: $json.body.budget,
  timeline: $json.body.timeline,
  source: 'website',
  status: 'nuevo'
}
```

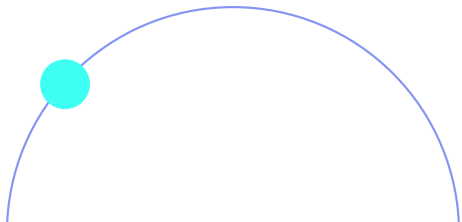
### Nodo 3: Supabase (Insert)

- *Operation:* Insert.
- *Table:* leads.
- *Data to Send:* All Defined Below.
- Mapea cada campo del Set node.

### Nodo 4: Postgres (Query Enriquecida)

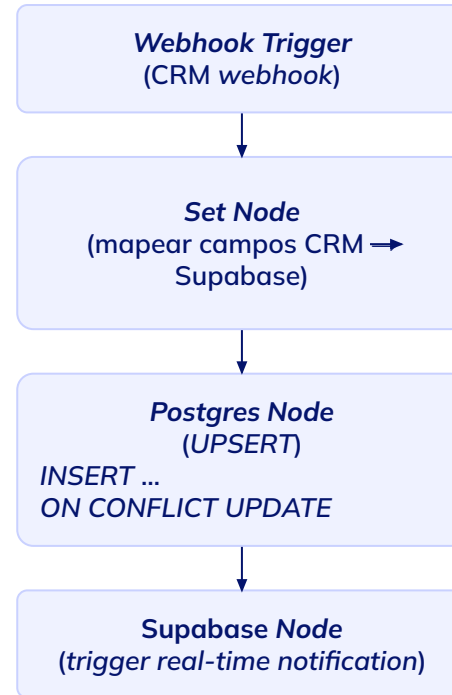
```
SELECT
  l.*,
  COUNT(a.id) as total_activities
FROM leads l
LEFT JOIN lead_activities a ON l.id = a.lead_id
WHERE l.email = $1
GROUP BY l.id
Parameters: ={{ $json.email }}
```

Esta combinación permite que el *workflow* no solo guarde datos, sino que también los use para tomar decisiones.



## Paso 4: Workflow Avanzado - Sincronización Bidireccional

**Escenario:** Cuando un *lead* cambia en CRM externo, actualizar en Supabase.



## Código del *Upsert*:

```
INSERT INTO leads (  
  email,  
  first_name,  
  last_name,  
  status,  
  assigned_to  
)  
VALUES (  
  $1, $2, $3, $4, $5  
)  
ON CONFLICT (email)  
DO UPDATE SET  
  first_name = EXCLUDED.first_name,  
  last_name = EXCLUDED.last_name,  
  status = EXCLUDED.status,  
  assigned_to = EXCLUDED.assigned_to,  
  updated_at = NOW()  
RETURNING *;
```

## Paso 5: Optimización con *Function Node*

Para lógica compleja, usa *Function Node*:

```
// Validación avanzada y transformación
const items = $input.all();
const validLeads = [];
const errors = [];
for (const item of items) {
  const email = item.json.email;
  // Validar email
  if (!email || !/^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/.test(email)) {
    errors.push({ email, error: 'Invalid email format' });
    continue;
  }
  // Validar campos requeridos
  if (!item.json.first_name || !item.json.last_name) {
    errors.push({ email, error: 'Missing required fields' });
    continue;
  }
  // Enriquecer datos
```

...

```
const enrichedLead = {
  ...item.json,
  email: email.toLowerCase(),
  full_name: `${item.json.first_name} ${item.json.last_name}`,
  score: calculateLeadScore(item.json), // función personalizada
  enriched_at: new Date().toISOString()
};
validLeads.push({ json: enrichedLead });
}
// Registrar errores si los hay
if (errors.length > 0) {
  $execution.getLastNodeExecutionData().push({
    json: { errors, total: errors.length }
  });
}
return validLeads;
// Helper function
function calculateLeadScore(lead) {
  let score = 0;
  if (lead.company_size === '201+') score += 30;
  if (lead.budget === '$5000+') score += 40;
  if (lead.timeline === 'urgent') score += 30;
  return score;
}
```